

Capstone Project Usage Instructions. Read it carefully.

- The project/assessment will be available for the full project/assessment window.
- You may start, pause, and continue your assessment anytime during this window by using the Save & Close button.
- When you click Save & Close, your progress will be saved and you can return later to continue the assessment
- After clicking Save & Close, if you log in again immediately, the Resume button may appear disabled for about 7-8 minutes.
- This is normal; please wait a few minutes for the Resume button to become active.
- Do not click Finish Test until you are completely done with your project.
- Once you click Finish Test, your assessment will be submitted permanently, and you will not be able to log in or make any further changes.

HealthBot: AI-Powered Patient Education System

MediTech Solutions is a leading healthcare technology company that provides software solutions to hospitals and clinics worldwide. They're looking to innovate in the field of patient education and engagement, recognizing that well-informed patients lead to better health outcomes and reduced healthcare costs.

The Challenge

MediTech Solutions has identified a critical gap in patient education. Many patients struggle to understand their medical conditions, treatment options, and post-treatment care instructions. This lack of understanding can lead to poor adherence to treatment plans, unnecessary hospital readmissions, and overall poorer health outcomes.

Your Role

As a Healthcare AI Specialist at MediTech Solutions, you've been tasked with developing a prototype for an AI-powered "HealthBot" that can provide personalized, on-demand health information to patients. This chatbot should be able to answer patient queries, provide summarized medical information, and assess patient understanding through simple quizzes.

The HealthBot Prototype

Your task is to create a LangGraph-based workflow for the HealthBot that includes the following functionality:

1. Ask the patient what health topic or medical condition they'd like to learn about.
2. Use the Tavily search engine (via LangChain community tool) to find relevant, up-to-date medical information on the topic.
3. Summarize the Tavily search results into a patient-friendly explanation.
4. Present the summarized information to the patient and allow them to read it.
5. Prompt the patient to indicate when they're ready for a comprehension check.
6. Generate a single, relevant quiz question based on the provided information.
7. Present the quiz question to the patient.
8. Allow the patient to enter their answer to the quiz question.
9. Evaluate the patient's response, providing a grade and explanation. The explanation should include relevant citations from the summary to reinforce learning.
10. Present the grade and explanation to the patient.
11. Ask if the patient would like to learn about another health topic or exit the session.
12. Either restart the flow for a new topic or end the session. Ensure that the state is reset when starting a new topic to maintain privacy and accuracy.

Goals and Impact

The HealthBot prototype aims to:

1. Improve patient understanding of medical conditions and treatments.
2. Provide 24/7 access to reliable health information.
3. Reduce the workload on healthcare providers for basic patient education.
4. Improve patient's engagement and empowerment in their own healthcare.
5. Potentially reduce hospital readmissions and improve treatment adherence.

If successful, this prototype could be expanded into a full-fledged product offering by MediTech Solutions, revolutionizing patient education in healthcare facilities worldwide.

Environment Setup

Your workspace will have the required libraries and appropriate versions installed, which can also be seen below:

1. Initialize uv

```
project folder >> pip install uv
```

```
project folder >> uv init
```

2. Create a virtual environment

```
project folder >> uv venv --python 3.11.13
```

3. Check the python version of your venv

```
project folder >> python --version
```

4. Activate the Virtual Environment On Windows (PowerShell):

```
project folder >> .\.venv\Scripts\Activate
```

5. Prepare requirements.txt

Create a file called requirements.txt in your project folder with:

6. Install the dependencies

```
(project folder) uv add -r requirements.txt folder)
```

7. show the dependencies

```
(project folder) pip list
```

Workspace Instructions

Ensure all project dependencies are installed.

Sign up for a free Tavily account, which will allow you to search the web and get back N pages and textual content for the pages, for free (first 1000 requests are free)

After signing up, you can get your API key here: [Tavily.com](https://tavily.com) (opens in a new tab)

Create a file named config.env in the same directory as the project notebook, and add entries for both OPENAI_API_KEY and TAVILY_API_KEY

e.g.

```
OPENAI_API_KEY="sk-*****"
```

```
TAVILY_API_KEY="tvly-*****"
```

Use python-dotenv to load the config.env file and ensure the appropriate env variables are set:

```
#Load in the OpenAI key and Tavily key.
```

```
# In the project folder, create a file named '.env'
```

```
# ensure your .env file contains keys named OPENAI_API_KEY="your key" and  
TAVILY_API_KEY="your key"
```

```
from dotenv import load_dotenv
```

```
import os
```

```
load_dotenv('config.env')
```

```
assert os.getenv('OPENAI_API_KEY') is not None
```

```
assert os.getenv('TAVILY_API_KEY') is not None
```

Project Instructions

- Create a LangGraph workflow that consists of nodes and edges that allow for the following functionality:
- Ask the patient what health topic or medical condition they'd like to learn about.
- Have the agent search Tavily using the LangChain community tool for Tavily, focusing on reputable medical sources.
- Have the agent summarize the Tavily results in patient-friendly language.
- Present the summarization to the patient and allow them to read it.
- Prompt the patient to indicate they are ready to take a comprehension check.
- Have the agent create a one question quiz based on the summarization of the health information.
- Present the quiz question to the patient.
- Prompt the patient to enter the answer to the quiz question.
- Have the agent grade the quiz question. The agent should respond with a grade, as well as a justification for why the grade was given, which should include citations from the health information summary.
- Present the grade and feedback to the patient.
- Ask the patient if they'd like to learn about a new health topic or if they want to exit the HealthBot session.
- Start the flow over again or exit. If the flow is started over again, you will need to reset the state so previous health information isn't carried over to maintain patient privacy and accuracy.

For patient input, you should use Jupyter notebook's input function, which will display a modal input with a description and text input to the patient. For displaying output to the patient, you can use the print function.

Rubric

Use this project rubric to understand and assess the project criteria.

1) Model and Tools LangGraph Configuration

Criteria	Submission Requirements
The student has properly configured OpenAI and Tavily so that the model is capable of calling Tavily search.	1. API keys are loaded correctly. 2. Successful API calls are made to both Tavily and OpenAI services. 3. OpenAI successfully calls Tavily for search.

2) Model Summarization of Search Results

Criteria	Submission Requirements
The student has correctly configured the model to summarize the search results from the Tavily tool.	1. The model has called Tavily for the desired learning subject 2. The model has received the Tavily search results 3. The model has been given a well written prompt to summarize the search results, using no other knowledge or data sources. 4. The model has created a 3-4 paragraph summarization of the search results

3) Model Quiz Question Creation and Grading Using Summarization

Criteria	Submission Requirements
The student has correctly configured the model to create a quiz using only the data from the summarization.	1. The model has been given a well written prompt to create a quiz question using only data from the summarization. 2. The quiz question should be answerable using the summary alone.
The student has correctly configured the model to grade the user's answer to the quiz question	1. The model should grade the user's answer to the quiz question, using only the summary as a data source. 2. The model should provide a grade (A, B, C, etc) as well as justification why it gave the grade that it did.

4) LangGraph State

Criteria	Submission Requirements
The student has correctly created a state object that is referenced and updated by nodes.	1. A state class is created with properties related to various data created, updated, and referenced by the different nodes. 2. The state is updated appropriately as each node is executed. a. Subsequent nodes can access state data set by previous nodes. 3. The model has access to previous messages (tool calls, summary, quiz question, etc)

5) LangGraph Nodes, Edges, and Modifications

Criteria	Submission Requirements
The student has correctly configured Nodes and Edges that allow for a learning workflow with human interaction to be executed.	1. Multiple nodes exist for the necessary steps in the workflow, such as search, user input, etc a. Nodes appropriately reference and update state. b. Ideally nodes will have a single responsibility, such as collecting user input, display output, etc. 2. Edges are configured in a way that brings the user through the sequential workflow nodes. a. The user should be able to restart the workflow or exit after the grade has been presented. 3. The entire workflow is executed and functions per the requirements a. The user is able to enter a subject, see the summary, take a quiz, see the quiz grade, and opt to learn about a new subject or exit